COMP 3311
DATABASE MANAGEMENT SYSTEMS

LECTURE 5
RELATIONAL DATABASE DESIGN: FUNCTIONAL DEPENDENCIES

RELATIONAL DATABASE DESIGN AND FUNCTIONAL DEPENDENCIES

- Relational database design requires that we find a "good" collection of relation schemas.

  ☞ **A bad design may lead to several problems.**

- Functional dependencies can be used to refine a relation schema reduced from an E-R schema by iteratively decomposing it (called normalization) to place it in a certain normal form.

Our focus →

  – The first four normal forms ⟹ use only functional dependencies.

  – Additional normal forms ⟹ use other types of dependencies (e.g., multi-valued dependencies, join dependencies)

FUNCTIONAL DEPENDENCY (FD): DEFINITION

Let R be a relation schema, X, Y be sets of attributes in R (i.e., R(X, Y), and *f* be a time-varying function from X to Y. Then

$$f: X \rightarrow Y$$

is a *functional dependency (FD)* if, *at every point in time*, for a given value of *x* in X there is ever at most one value of *y* in Y.

Example: PGStudent(studentId, name, supervisor, specialization)

f: supervisor → specialization

Consequences of the FD *f*:

- If two student tuples have the same supervisor (e.g., Papadias), then they must have the same specialization (e.g., Database).
- On the other hand, if two student tuples have different supervisors, then they may have the same specialization or different specializations.

PGStudent

studentId	name	supervisor	specialization
23455789	Bruno Ho	Yang	Artificial Intelligence
23556789	Jenny Jones	Papadias	Database
25678989	Kathy Ko	Kim	Software Technology
26789012	Susan Sze	Papadias	Database
26184624	Terry Tam	Song	Artificial Intelligence
26186666	Carol Chen	Tai	Graphics

FUNCTIONAL DEPENDENCY (FD): DEFINITION

- We normally omit the *f:* and simply write the FD as $X \rightarrow Y$.
  - X is called the determinant set or left-hand side (LHS) of the FD.
  - Y is called the dependent set or right-hand side (RHS) of the FD.

    ☞ **We say that X determines Y or Y depends on X.**

- In general, X (and Y) can be sets of attributes.
  E.g., if $X \equiv X_1 \cup X_2 \cup \ldots \cup X_n$, then we can write $X_1X_2\ldots X_n \rightarrow Y$

- A functional dependency $X \rightarrow Y$ is trivial if $Y \subseteq X$.

  ☞ **Y appears on both the LHS and the RHS of the FD.**

  ➢ Trivial FDs hold for all relation instances.

- A functional dependency $X \rightarrow Y$ is non-trivial if $Y \cap X = \varnothing$.

  ☞ **Y does not appear on both the LHS and the RHS of the FD.**

  ➢ Non-trivial FDs are given as constraints when designing a database.

  ➢ Non-trivial FDs constrain the set of legal relation instances.

FUNCTIONAL DEPENDENCIES AND KEYS

- An FD is a generalization of the concept of a *(candidate) key*.

  For the relation

  PGStudent (studentId, name, supervisor, specialization)

  we can write:

  studentId → name, supervisor, specialization

  because the (candidate) key, studentId, determines the (single) value of all the attributes (i.e., the entire tuple).

  ☞ Given a value for studentId there will ever be at most one value for name, supervisor and specialization.

- If two tuples in the PGStudent relation have the same studentId, then they must have the same values on all attributes.

  ☞ **They must be the same tuple!**

  (Since the relational model does not allow duplicate tuples.)

INFERENCE RULES FOR FDS

- Given a set of functional dependencies F, there are certain other functional dependencies that are *logically implied* by F.
- We can find all functional dependencies implied by F by (repeatedly) applying the following inference rules for FDs.

IR1: Reflexivity	If $Y \subseteq X$, then $X \rightarrow Y$ (*trivial FD*)	Armstrong's Axioms (basic rules)
IR2: Augmentation	$X \rightarrow Y \models XZ \rightarrow YZ$	
IR3: Transitivity	$X \rightarrow Y, Y \rightarrow Z \models X \rightarrow Z$	
IR4: Union	$X \rightarrow Y, X \rightarrow Z \models X \rightarrow YZ$	Additional rules (derivable from IR1, IR2 and IR3)
IR5: Decomposition	$X \rightarrow YZ \models X \rightarrow Y$ and $X \rightarrow Z$	
IR6: Psuedotransitivity	$X \rightarrow Y, WY \rightarrow Z \models WX \rightarrow Z$	

INFERENCE RULES FOR FDS: EXAMPLES

IR1: Reflexivity If $Y \subseteq X$, then $X \rightarrow Y$ (*trivial FD*)

name ⟶ name	since name ⊆ name
name, supervisor ⟶ name	since name ⊆ {name, supervisor}
name, supervisor ⟶ supervisor	since supervisor ⊆ {name, supervisor}

IR2: Augmentation $X \rightarrow Y \models XZ \rightarrow YZ$

studentId ⟶ name	(given)
studentId, supervisor ⟶ name, supervisor	(inferred using IR2)

IR3: Transitivity $X \rightarrow Y, Y \rightarrow Z \models X \rightarrow Z$

studentId ⟶ supervisor	(given)
supervisor ⟶ specialization	(given)
studentId ⟶ specialization	(inferred using IR3)

INFERENCE RULES FOR FDS: CLOSURE

Inference rules IR1, IR2 and IR3 are sound and complete.

sound: Given a set of FDs, F, specified on a relation schema **R**, any FD that we can infer from F by using IR1 to IR3 *will hold* in every relation instance **r** of **R** that satisfies F (i.e., it is a *true FD*).

complete: Using IR1, IR2 and IR3 repeatedly to infer FDs will infer *all* the FDs that can be inferred from F. (i.e., there are *no other FDs* that are true).

The set of all functional dependencies *logically implied* by F is called the closure of F denoted as F^+.

EXAMPLE OF FDS IN THE CLOSURE F^+

Given: R(A, B, C, G, H, I)

F = {A→B
A→C
CG→H
CG→I
B→H}

IR1: Reflexivity	If $Y \subseteq X$, then X→Y
IR2: Augmentation	X→Y ⊨ XZ→YZ
IR3: Transitivity	X→Y, Y→Z ⊨ X→Z
IR4: Union	X→Y, X→Z ⊨ X→YZ
IR5: Decomposition	X→YZ ⊨ X→Y and X→Z
IR6: Psuedotransitivity	X→Y, WY→Z ⊨ WX→Z

Some additional members of F^+ (derived using inference rules)

A→H
AG→I
CG→HI

ATTRIBUTE CLOSURE

- The closure of X under F (denoted by X^+) is the set of attributes that are functionally determined by X under F.

$X \rightarrow Y$ is in $F^+ \Leftrightarrow Y \subseteq X^+$

X is a set of attributes

Given studentId (e.g., studentId is X).

If studentId $\rightarrow$ name is in F^+
then name is in studentId$^+$
(i.e., studentId$^+$= {studentId, name, …})

Why is studentId in X^+?

If studentId $\rightarrow$ supervisor is in F^+
then supervisor is in studentId$^+$
(i.e., studentId$^+$= {studentId, name, supervisor, …})

ATTRIBUTE CLOSURE: ALGORITHM

Input:

R a relation schema
F a set of functional dependencies
$X \subset R$ (the set of attributes for which we want to compute the closure)

Output:

X^+ the closure of X w.r.t. F

$X^{(0)} := X$
Repeat
  $X^{(i+1)} := X^{(i)} \cup Z$, where Z is the set of attributes such that there exists
    $Y \rightarrow Z$ in F, and $Y \subseteq X^{(i)}$
Until $X^{(i+1)} := X^{(i)}$
Return $X^{(i+1)}$

☞ **For every set of attributes Y in X^i, if Y is the LHS of an FD, then add the RHS attributes Z to the closure; repeat until there are no more attributes to add.**

ATTRIBUTE CLOSURE: USES

Testing for Superkey

To test if X is a superkey, compute X^+, and check if X^+ contains all attributes of **R**. If X is minimal, then it is a candidate key.

☞ An attribute that is part of *any* candidate key is a prime attribute; otherwise, it is a non-prime attribute.

Testing Functional Dependencies

To determine whether a functional dependency $X \rightarrow Y$ holds (i.e., if $X \rightarrow Y$ is in F^+), compute X^+ and check if $Y \subseteq X^+$.

Computing the Closure of F

For each subset $X \subseteq \mathbf{R}$, compute X^+ and, for each $Y \subseteq X^+$, output a functional dependency $X \rightarrow Y$.

ATTRIBUTE CLOSURE: EXAMPLE

R(A, B, C, D, E, G)

F = { AB→C, C→A, BC→D, ACD→B,
D→EG, BE→C, CG→BD, CE→AG }

☞ Compute the closure of BD w.r.t F

$BD^{(0)}$ = {B, D}

$BD^{(1)}$ = {B, D, E, G} apply D→EG add E, G to the closure of BD

$BD^{(2)}$ = {B, C, D, E, G} apply BE→C add C to the closure of BD

$BD^{(3)}$ = {A, B, C, D, E, G} apply C→A add A to the closure of BD

$BD^{(4)} = BD^{(3)}$ no more FDs can be applied

$\{B, D\}^+ = \{A, B, C, D, E, G\}$ **Is BD a candidate key?**

REDUNDANCY OF FDS

- Sets of functional dependencies may have redundant functional dependencies that can be inferred from the other FDs.

  A→C is redundant in: {A→B, B→C, A→C} **Why?**

- Parts of a functional dependency may be redundant.

  Example of extraneous/redundant attribute in RHS.

  { A→B, B→C, A→CD } can be simplified to
  { A→B, B→C, A→D } **Why?**

  Example of extraneous/redundant attribute in LHS (partial dependency).

  { A→B, B→C, AC→D } can be simplified to
  { A→B, B→C, A→D } **Why?**

CANONICAL COVER

A *canonical cover* for F is a set of functional dependencies F_c such that

- F and F_c are equivalent (i.e., they have the same closure F^+).
- F_c contains no redundancy (i.e., no extraneous attributes).
- The LHS of each functional dependency in F_c is unique.

☞ **Every set of FDs has a canonical cover.**

☞ **The canonical cover for a set of FDs is not unique.**

☞ **Testing for FD violation using F_c is usually simpler.**

CANONICAL COVER: ALGORITHM

☞ **Recall that two FDs X→Y, X→Z, can be converted to X→YZ (IR4: Union).**

Algorithm to compute the canonical cover of F.

1. $F_c = F$
2. Repeat:
   a) Use the union rule (IR4) to replace any FDs in F_c of the form X→Y and X→Z with X→YZ.
   b) Find an FD X→Y in F_c with an extraneous attribute either in X or in Y.
   If an extraneous attribute is found, delete it from X→Y in F_c.
3. until F_c does not change.

☞ **The union rule may become applicable after some extraneous attributes have been deleted, so it must be reapplied.**

CANONICAL COVER: EXAMPLE COMPUTATION

R(A, B, C)

F = {A→BC, B→C, A→B, AB→C}

- Use union rule (IR4) to combine A→BC and A→B into A→BC

  F_c is now {A→BC, B→C, AB→C}

- Delete extraneous attribute A in AB→C because of B→C.

  F_c is now {A→BC, B→C}

- Delete extraneous attribute C in A→BC because using A→B and B→C we can infer A→C.

  F_c is now {A→B, B→C}

☞ The canonical cover F_c = {A→B, B→C}